STM32 with system peripherals

Author: Brian Kang
Create: 4/26/2026
Last Modified: 4/26/2026

MCU means MCP (processor) with embedded RAM, Flash, and peripherals.

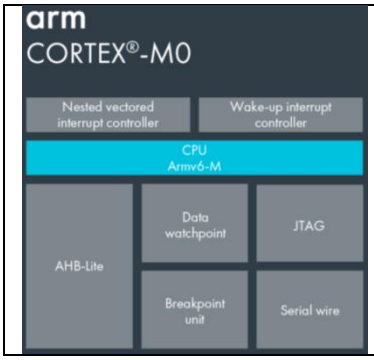
	Description
Cortex-A	**Application** processor cores for performance-intensive systems
Cortex-R	High-performance cores for **real-time** applications
Cortex-M	**Microcontroller** cores for a wide range of embedded applications

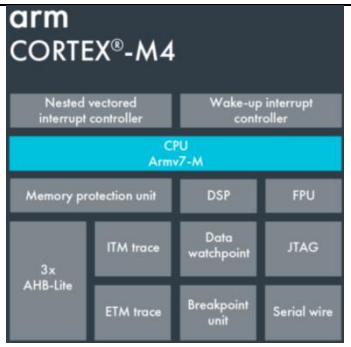
Experienced Core

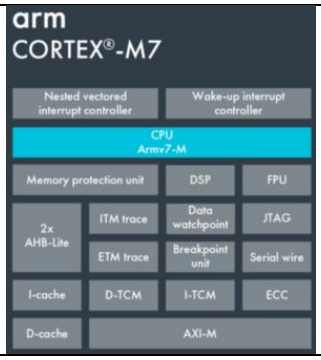
Core	bit		Target
MIPS	32	with Rom Writer	DVD player
Soju	16	small private RISC with JTAG	Control multi-DPS
Leon3	32	with Yamon. Security Camera System.	Network DVR
DP390	8	Enhanced 8052. with uVision Professional.	Scaler+Panel Controller
Cortex-A9	32	NXP iMX6. Quad. 1.2GHz	Sabre, Wand board
Cortex-A53	32/64	4x Cortex-A53. TW9992 and ISL79987	Raspberry Pi 3.
Cortex-57 Cortex-53	64	Drivers for tw9992,isl79987, raa278846, raa278842, raa279972 and raa279974. Capture and display application, cam-thread for 4CH, multi-thread, OpenGL, Wayland, DRM and SurroundView.	R-CAR H3SK R-CAR V3SK

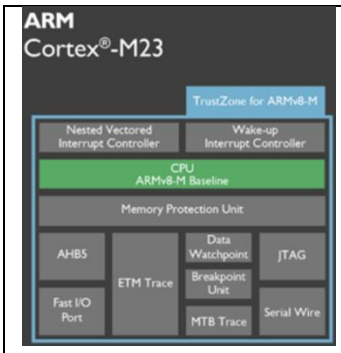
1. STM32 MCU

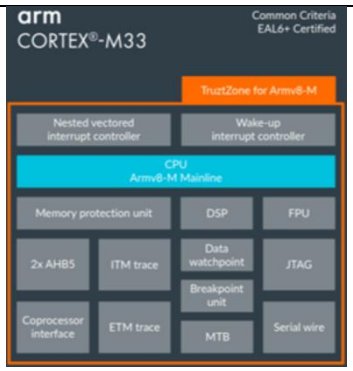
1.1 Cortex-M family

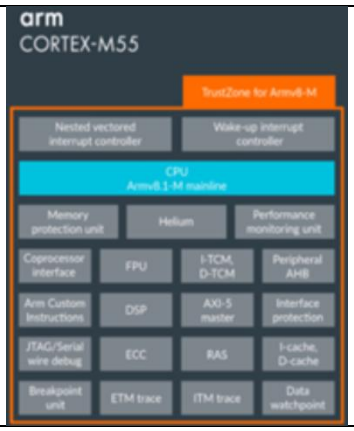












Cortex-M4 components

	Description
CPU	Armv7-M
NVIC	Nested Vectored Interrupt Controller
MPU	Memory Protection Unit. No MMU(for Virtual Memory). 8 units
FPU	Floating Point Unit
DSP	Digital Signal Processor
BUS	AHB Amba Bus

Group

	STM32XX
High Performance	F2, F4, F7, H5, H7, N6
Mainstream	C0, G0, F0, F1, G4, F3
Ultra low power	L0, U0, L4, L4+, L5, U5, U3
Wireless	WL, WB0, WB, WBA

Key Hardware Features

What makes STM32 stand out is the "peripheral" set. Most chips include:

Communication	I2C, SPI, UART, CAN (and CAN-FD), USB, and Ethernet.
Analog	High-speed ADCs (up to 12-bit or 16-bit), DACs, and built-in operational amplifiers.
Timers	Advanced timers for PWM (motor control), input capture, and encoder interfaces.
DMA	Allows data to move between peripherals and memory without taxing the CPU, which is crucial for high-speed data handling.

1.2 STM32 Board

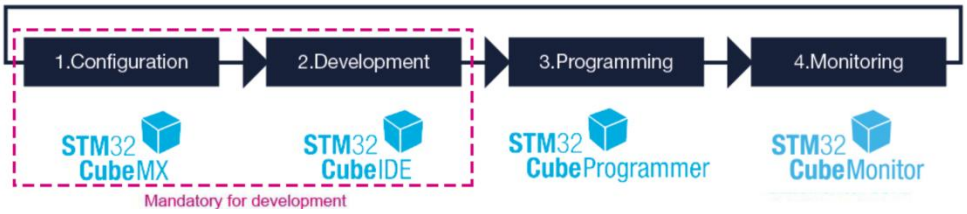
STM32 Nucleo	STM32 Discovery kit	STM32 Eval board

 Cheaper, about $17. **NUCLEO-144** STM32F767ZI EVAL BRD. SL-Link (for in-circuit debugger) is included.	 Extension Connector + Display $30 $100 with 4.3” LCD $190 with LCD+Camera	 $405~$500

Arduino is deprecated

Speed	Clock speeds start around 48 MHz and go up to 550 MHz (vs. 16 MHz for an Arduino Uno).
Multitasking	Better support for Real-Time Operating Systems (RTOS) like FreeRTOS or Azure RTOS.
Concurrency	Features like DMA and multiple buses allow the chip to do several things at once.

1.3 Software Tools



	Description
STM32CubeMX	A graphical tool that lets you configure your pins, clocks, and peripherals. It generates C initialization code automatically
STM32CubeIDE	An all-in-one Eclipse-based IDE for coding and debugging.
STM32CubeProgrammer	For flashing the firmware onto the hardware.
Hardware	The **Nucleo** boards are the best starting point—they are affordable, include an integrated debugger (ST-LINK), and are Arduino-shield compatible.

Third-party tool-chains

For their superior optimization, safety certifications, or advanced debugging capabilities.

IAR Embedded Workbench for Arm	IAR is often considered the "gold standard" for professional embedded development, particularly in automotive and medical industries. **The Compiler:** Its proprietary C/C++ compiler often produces significantly smaller and faster binaries compared to the standard GCC used in CubeIDE. **Safety Critical:** It comes with functional safety certifications (IEC 61508, ISO 26262) required for mission-critical software. **C-SPY Debugger:** Provides extremely detailed power debugging and complex code

	coverage analysis. **Cost:** It is a commercial product with a high licensing fee.
Keil MDK (Microcontroller Development Kit)	Owned by Arm themselves, Keil is the most popular professional IDE for Arm-based chips. **µVision IDE:** Includes a very powerful debugger and simulator. **Middleware:** Comes with "MDK-Professional," which includes pre-validated stacks for TCP/IP, USB, File Systems, and GUI. **Cortex Microcontroller Software Interface Standard (CMSIS):** Since it's an Arm product, it has the tightest integration with CMSIS, making it easy to port code between different Cortex-M vendors.
Segger Embedded Studio (SES)	A fast, C/C++ IDE that has gained massive popularity recently. **The Segger Link:** It is designed by the makers of the **J-Link** debugger (the industry-standard probe). **Performance:** It is known for being extremely fast at indexing and building large projects compared to Eclipse-based tools. **Licensing:** It is free for non-commercial use and for use with certain silicon vendors (like Nordic), but requires a license for professional STM32 development.
VS Code + STM32 VS Code Extension	If you prefer a modern, lightweight coding environment over a "heavy" traditional IDE, this is the modern trend. **Official Support:** ST now provides an official "STM32 VS Code Extension" that handles the integration between VS Code, the compiler, and the debugger. **Workflow:** You still use **STM32CubeMX** to generate the project, but you select "CMake" as the toolchain and open the folder in VS Code. **Flexibility:** It allows you to use your favorite extensions for Git, AI-copilots, and custom themes.
CLion (by JetBrains)	For developers coming from a web or desktop background, CLion offers the most advanced code completion and refactoring tools. **Refactoring:** Unmatched "IntelliSense" and code analysis features. **STM32 Integration:** It has built-in support for STM32CubeMX projects and uses OpenOCD for debugging.

1.4 HW probe (debugger)

ST-LINK	The standard, low-cost probe (built into Nucleo boards). So, you need only USB mini-B.
Segger J-Link	The professional choice, known for incredibly fast flash speeds and "Unlimited Flash Breakpoints."
IAR I-jet	Specifically designed to integrate with IAR for high-speed trace and power profiling.

2. First Project : Blink LED

ref: STM32StepByStep:Step2 Blink LED - stm32mcu

2.1 Blinking LED with STM32CubeMX and HAL

Learn how to use STM32CubeMX tool to configure the pins, start the peripherals, build and generate your starting projects with initialization C code using HAL.

2.1.1 Create Project

Run STM32CubeMX tool.
Click New Project or Menu -> File -> New Project.
From Board Selector section, filter to select and use NUCLEO-L476RG board.
(Order code:NUCLEO-L476RG Board reference:MB1136 Manual:UM1724 Target STM32:**STM32L476RGT6**)

Check **Nucleo64** type.
Check **STM32L4** on MCU Series.
Select **NUCLEO-L476RG** board using Board selector:

Click Start Project to continue.
Answer Yes Initialize all peripherals with their default Mode ? popup.

2.1.2 Pinout Configuration

Verify in *Pinout* tab, under *SYS peripheral*, that *Serial Wire* is selected as Debug interface:
The corresponding pins PA13 and PA14 are assigned and configured automatically.
This example shows the use of the green LED pin LD2 present on the NUCLEO-L476RG board as GPIO_Output.
To verify that LD2 is set to GPIO_Output mode:

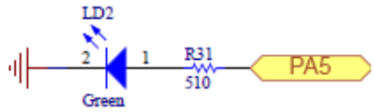
Type “LED” in the Find field and check that LD2 (green Led) is enabled to **PA5** pin as GPIO_Output.
When found, the pin that matches the search criteria blinks on the Chip view.
Click on Chip view to stop the blinking. The signals can be set directly from the pinout view:

2.1.3 Clock Configuration

In the Clock Configuration tab, check that STM32CubeMX automatically configures the internal oscillator in the clock system with PLL @80MHz and proposes the PLL configuration as follows:
1. HSI selected in PLL Source Mux (HSI – High Speed Internal clock)
2. PLLCLK selected in the System Clock Mux
3. HCLK set to **80**

2.1.4 GPIO Configuration

To configure the GPIOs, click the GPIO button in the Configuration Tab to open the GPIO Configuration window.
In the GPIO Tab, select Pin Name column PA5 to display the corresponding GPIO parameters and configuration to drive the NUCLEO-L476RG LED:



GPIO Output level: it is set to Low by default and can be changed to High.
GPIO mode automatically configures the pins with the relevant alternate function and GPIOs into Output Push Pull mode.
GPIO Pull-up/Pull-dow set to No pull-up and no pull-down by default can be configured when other choices are allowed.
GPIO Maximum output speed set to Low by default for the power consumption optimization can be changed to a higher frequency to fit the application requirements.
User Label is a name assigned to a GPIO. The GPIO can be found under this name via the Find menu.
Click *Apply* then *OK* to close the window.

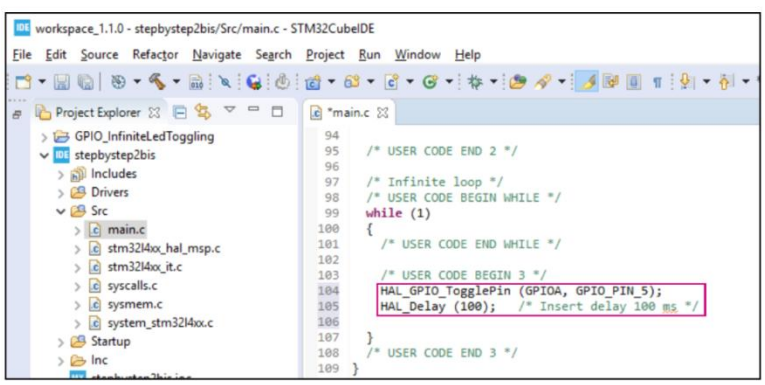
2.1.5 Configure project and generate source code

Switch to the Project Manager tab to configure the project. In the Project tab:

Fill the Project Name and Project Location fields
Set Toolchain/IDE to STM32CubeIDE.
In the Code Generator tab, ensure that the following options are checked:
- In STM32Cube Firmware Library Package section: Copy all used libraries into the project folder.
- In Generated files section: Keep user code when regenerating the C code option, which only applies to the user sections within the STM32CubeMX generated files.

To generate the project in STM32CubeIDE:
- Click on ***Generate Code***.
- Click on ***Open Project*** to open the project with STM32CubeIDE.

2.1.6 Add Code




```
#include “main.h”

#include <stdio.h>
```

```
main()
{
  SystemClock_Config();

  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_TIM16_Init();

  // Start timer
  HAL_TIM_Base_Start(&htim16);

  // Get current time (microseconds)
  timer_val = __HAL_TIM_GET_COUNTER(&htim16);

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
#if 1
    // If enough time has passed (1 second), toggle LED and get new timestamp
    if (__HAL_TIM_GET_COUNTER(&htim16) - timer_val >= 10000)
    {
      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
      timer_val = __HAL_TIM_GET_COUNTER(&htim16);
    }
#else
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(100); /* Insert 100ms delay */
#endif
  }
}
```

2.1.7 Build the project

To power the NUCLEO-L476RG, use its CN1 connector to connect in with a computer through a USB Type-A to Mini-B cable.
Click on the project from the project explorer, then right click and select ***Build Project***, to compile the project (or click on ***Build button*** on the toolbar).

2.1.8 Debug the project

Click on the ***Build*** toolbar icon , then on the ***Debug*** button to build the project and start the debug session (or in the menu, select ***Project*** > ***Build Project*** then ***Run*** > ***Debug***).

Click on ***Resume*** icon to continue the execution.
Now watch the green LED (LD2) toggling on the Nucleo-L476RG board.

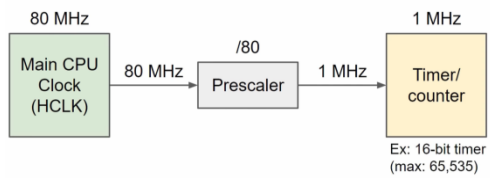
2.2 Timer

Usage:

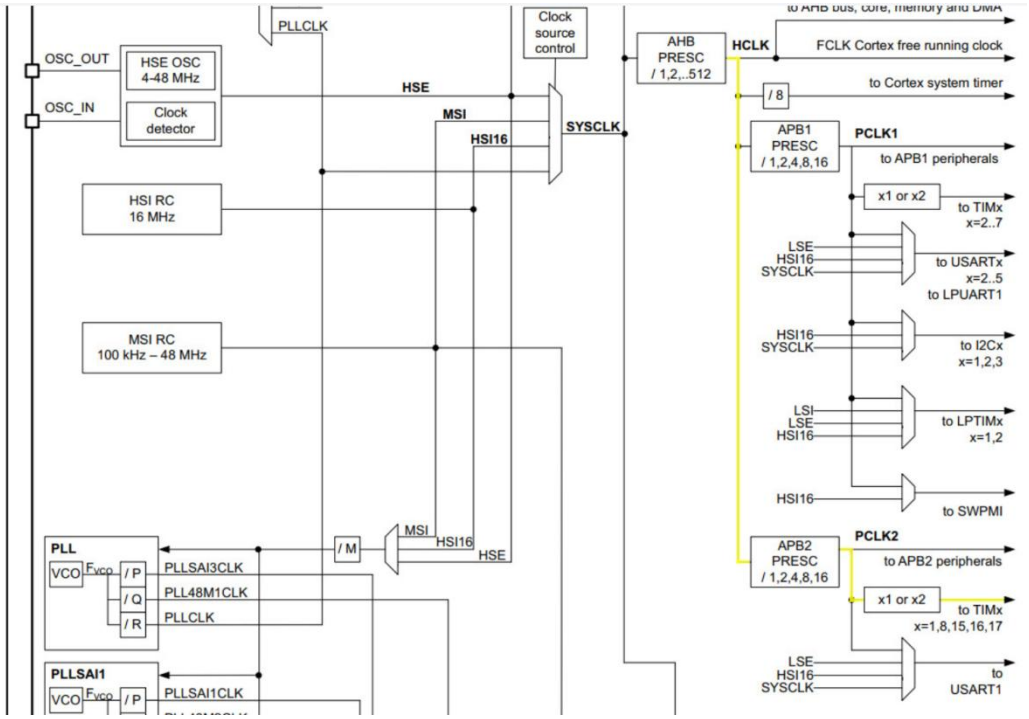
- Output Compare (OC),
- Input Capture (IC)
- PWM. By adjusting the on versus off time (duty cycle)

Prescaler

: 80MHz (/80) => 1MHz



Clock Tree



STM32CubeIDE. Timer => TIM16.

: Set the Prescaler(PSC) to 79. (for 80).

: Set counter value (65536-1)

Read value at 0x24 TIMx_CNT

```
static void MX_TIM16_Init(void)
{
  ..
  /* USER CODE END TIM16_INIT 1 */
  htim16.Instance = TIM16;
  htim16.Init.Prescaler = 80 -1;
  htim16.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim16.Init.Period = 65536 -1;
  htim16.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim16.Init.RepetitionCounter = 0;
```

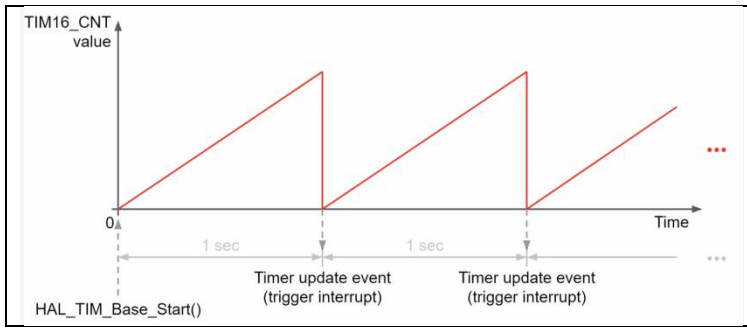
```
  htim16.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if(HAL_TIM_Base_Init(&htim16) != HAL_OK) {
    Error_Handler();
  }
  ..
}
```

```
 MX_TIM16_Init(); //it will use our selection for TIM16 timer.

 HAL_TIM_Base_Start(&htim16);

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    // Get current time (microseconds)
    timer_val = __HAL_TIM_GET_COUNTER(&htim16);

    // Wait for 50 ms
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    HAL_Delay(50);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

    // Get time elapsed
    timer_val = __HAL_TIM_GET_COUNTER(&htim16) - timer_val;

    // Show elapsed time
    uart_buf_len = sprintf(uart_buf, "%u us\r\n", timer_val);
    HAL_UART_Transmit(&huart2, (uint8_t *)uart_buf, uart_buf_len, 100);

    // Wait again so we don't flood the Serial terminal
    HAL_Delay(1000);

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
```

You should see that it takes about 51000 microseconds to execute HAL_Delay(50) (interestingly, it seems that HAL_Delay() adds a millisecond).

2.3 Timer Interrupt

We will set the Timer 16 prescaler to be 8000 so that it ticks at a rate of 10 kHz.
We will then set its maximum value to 10,000 – 1.




```
// Start timer
  HAL_TIM_Base_Start_IT(&htim16); /* with Interrupt */
```

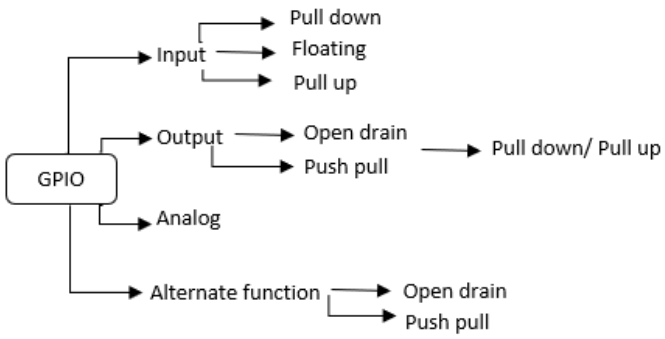
```
/* USER CODE BEGIN 4 */

// Callback: timer has rolled over
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  // Check which version of the timer triggered this callback and toggle LED
  if (htim == &htim16 )
  {
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  }
}

/* USER CODE END 4 */
```

This is an interrupt handler.

In that definition, we check to make sure that the timer handle (htim) is indeed our Timer 16 and then toggle the LED pin. Note that this is a generic timer interrupt callback. If you set up multiple timer interrupts, this one callback will be called for any of them, which is why we check the timer instance handle (htim) to differentiate among the possible timers.

2.4 GPIO

Pin Mode	Each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes: - input or output - analog - alternate function (AF).
Pin characteristics	***Input*** : no pull-up and no pull-down or pull-up or pull-down ***Output*** : push-pull or open-drain with pull-up or pull-down capability ***Alternate function*** : push-pull or open-drain with pull-up or pull-down capability.

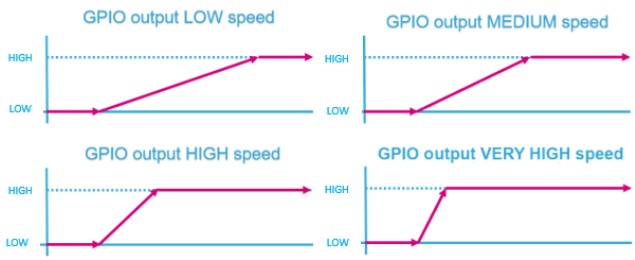


GPIO (pin) output-speed configuration

Change the rising and falling edge when the pin state changes from high to low or low to high.

A higher GPIO speed increases the EMI noise from STM32 and increases the STM32 consumption.

It is good to adapt the GPIO speed to the peripheral speed. For example, **low** speed is optimal for toggling GPIO at 1 Hz, while using SPI at 45 MHz requires ***very high*** speed setting.



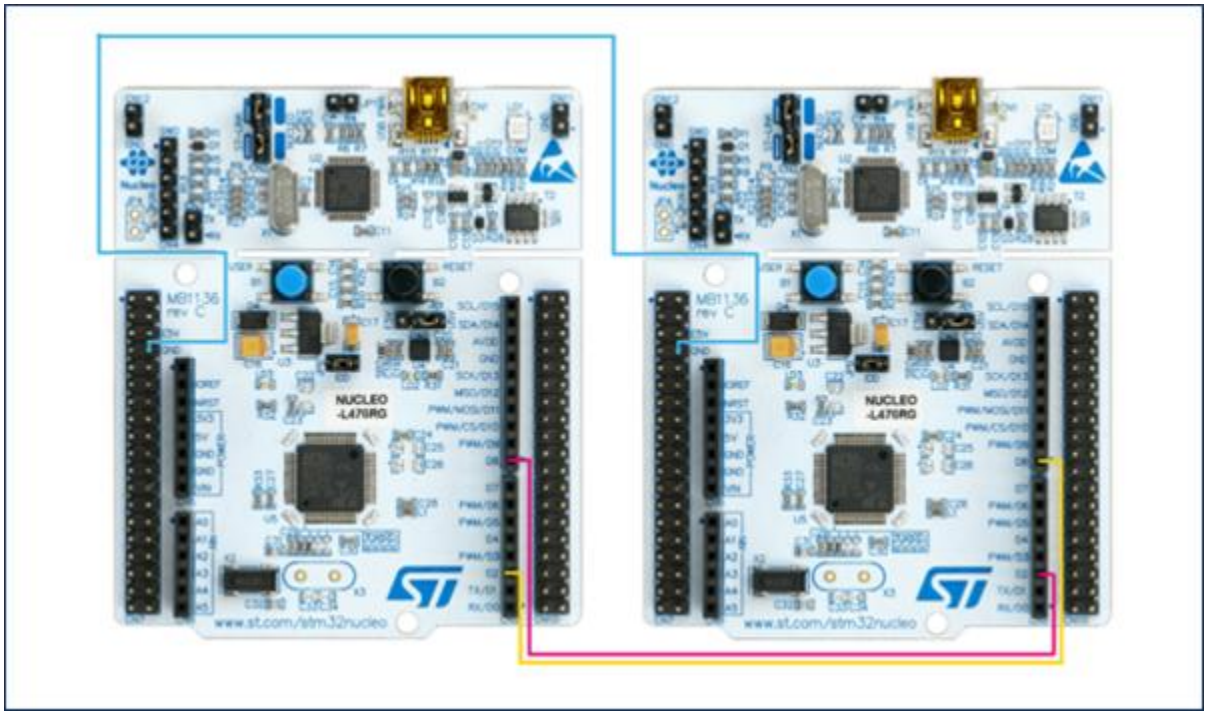
3 Peripheral - UART

UART using two signal lines Tx and RX and one Ground line. TX/RX between two machines has to be crossed.

Board 1 RX connected to board 2 TX Board 1 TX connected to board 2 RX Board 1 GND connected to board 2 GND

Second thing is a configuration both machine. “115200 8N1” is a normal notation that means 115200 Baud, 8 Data bit, No Parity, 1 Stop bit. And need to disable Flow Control.

STM32 tutorial suggests using two machines that is good to explain about Tx and Rx without other consideration.



On the real environment, use “USB2Serial adapter”, and Use PN2:RxD, PIN3:TxD and Pin5:GND

	Tx function	Rx function
Blocked Mode	HAL_UART_Transmit	HAL_UART_Receive
Interrupt Mode	HAL_UART_Transmit_IT	HAL_UART_Receive_IT
DMA Mode	HAL_UART_Transmit_DMA	HAL_UART_Receive_DMA

On Nucleo board	**PA9** for transmission (TX)	**PA10** for reception (RX)
On RS232 connector (Note: Need to cross)	**PIN3**	**PIN2**

	API Function
Blocked Mode	HAL_UART_Transmit(), HAL_UART_Receive()
Interrupted Mode	HAL_UART_Transmit_IT(),HAL_UART_Receive_IT()
DMA mode	HAL_UART_Transmit_DMA(),HAL_UART_Receive_DMA()

3.1 GPIO & UART Configuration

Set USART1 mode to **Asynchronous mode**

Configure **Parameter Settings** as shown in the figure below:

Basic

Baud Rate: 115200 Bit/s

Word Length: 8bit

Parity: None

Stop Bits: 1

Advanced

Data Direction: Receive and Transmit

Over Sampling: 16 Samples

Single Sample: Disable

To select the correct GPIOs pins, it is necessary to refer to the datasheet. Look for the alternate function table. STM32L476 USART1 uses **PA9** for transmission (TX) and **PA10** for reception (RX) as shown below:

3.2 Blocked Mode

Tx example	Rx example
/* USER CODE BEGIN 0 */ uint8_t tx_buff[]={0,1,2,3,4,5,6,7,8,9}; /* USER CODE END 0 */	/* USER CODE BEGIN 0 */ uint8_t rx_buff[10]; /* USER CODE END 0 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */
 	
  /* USER CODE BEGIN 3 */
 	
  HAL_UART_Transmit(&huart1, tx_buff, 10, 1000);
  HAL_Delay(10000);
 	
}
/* USER CODE END 3 */
	/* Infinite loop */
	/* USER CODE BEGIN WHILE */
	while (1)
	{
	  /* USER CODE END WHILE */
	
	  /* USER CODE BEGIN 3 */
	  //if transfer is successful
	  if(HAL_UART_Receive(&huart1,
	  rx_buff, 10, 1000)==HAL_OK)
	  {
	  //You need to toggle a breakpoint on this line!
	  __NOP();
	  } else {
	  __NOP();

	} } /* USER CODE END 3 */

3.3 with Interrupt

It uses functions that has _IT prefix.

Enable USART1 global interrupt.

Tx	Rx
/* USER CODE BEGIN 0 */ //ABCDEFGHIJ in ASCII code uint8_t tx_buff[]={65,66,67,68,69,70,71,72,73,74}; /* USER CODE END 0 */	/* USER CODE BEGIN 0 */ uint8_t rx_buff[10]; /* USER CODE END 0 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */
 	
  /* USER CODE BEGIN 3 */
 	
  HAL_UART_Transmit_IT(&huart1, tx_buff, 10);
  HAL_Delay(10000);
 	
}
/* USER CODE END 3 */
	/* USER CODE BEGIN 2 */
	HAL_UART_Receive_IT(&huart1, rx_buff, 10);
	/* USER CODE END 2 */
	
	/* USER CODE BEGIN 4 */
	void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
	{
	  HAL_UART_Receive_IT(&huart1, rx_buff, 10);
	  //You need to toggle a breakpoint on this line!
	}
	/* USER CODE END 4 */

3.4 Other method is using DMA.

Open the UART.ioc file in the STM32CubeIDE project as shown in the figure below:

Add the DMA request as shown in the figure below:

DMA Request	Channel	Directopn	Priority
USART1_RX	DMA1 Channel 5	Peripheral to Memory	Low
USART1_TX	DMA1 Channel 4	Memory to Peripheral	Low

'Ctrl+S' to generate the project.

Tx	Rx
/* USER CODE BEGIN 0 */ //ABCDEFGHIJ in ASCII code uint8_t tx_buff[]={65,66,67,68,69,70,71,72,73,74}; /* USER CODE END 0 */	/* USER CODE BEGIN 0 */ uint8_t rx_buff[10]; /* USER CODE END 0 */
/* Infinite loop */ /* USER CODE BEGIN WHILE */	/* USER CODE BEGIN 4 */ void HAL_UART_RxCpltCallback(UART_HandleTypeDef

while (1)
{
  /* USER CODE END WHILE */
 	
  /* USER CODE BEGIN 3 */
 	
  HAL_UART_Transmit_DMA(&huart1, tx_buff, 10);
  HAL_Delay(10000);
 	
}
/* USER CODE END 3 */
	*huart)
	{
	  //You need to toggle a breakpoint on this line!
	  HAL_UART_Receive_DMA(&huart1, rx_buff, 10);
	}
	/* USER CODE END 4 */

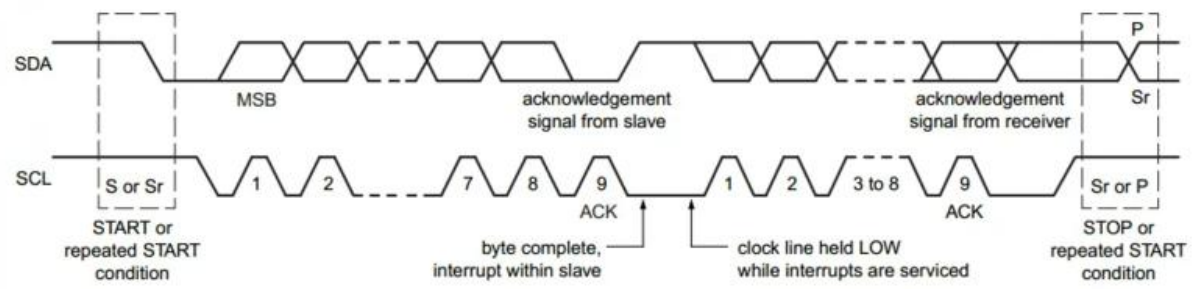
3.5 Prepare Prompt

We can implement the command interface between your Host and Necleo board. See below example.

```
Boot STM32 firmware x.xx
..
Hello World!
>
> mode i2c
Enter I2C mode. current I2C ID is 0x88(0x44 on 7bit)
i2c_88> help
id [xx]   - set I2C ID
r idx     - read indexed register value. ex: r 0
w idx data – write value on indexed register. ex: w 1 ff
b idx start_bit end_bit data – update bits. ex: b 2 7 4 0F to change high nibble to “1111b” on register 2.
d [start stop] – read register values from Start to End.
exit - exit i2c mode
i2c_88> exit
Leaving I2C mode..
>
```

4 Peripheral – I2C

Mode	Speed	Pull-up (Common is 47kOhm)	Max Rise Time(tr)
Standard-Mode (Sm)	up to 100 kbit/s	4.7kOhm ~ 10KOhm	1000ns
Fast-Mode (Fm)	up to 400 kbit/s	2.2kOhm~4.7KOhm	300ns
Fast-Mode Plus (Fm+)	up to 1 Mbit/s	1KOhm~2.2kOhm	120ns



	Description
S	Start or ReStart SCL:High SDA:move to Low.
Sr	Repeated Start. Same as S.
P	Stop SCL:High SDA:move to High.
Others	Without S and P, all SDA signal changes on SCL Low state. MSB first.
I2C ID	Use 7bit with 0 for Write, and 1 for Read.
Ack Nack	Each 8 bit data, Ack/Nack is generated. Master release SDA. SDA go to High by Pull-Up register. Client move to SDA low to make Ack. If Client can not make SDA low, it means Nack.
Write	S + I2C_ID + 0(W) + I2C Register Index + Data + .. + P
Read	S + I2C_ID + 0(W) + I2C Register Index + Sr + I2C_ID+1(R) + Data + .. + P

Note: On auto index increase mode (default), index can be increased to 255.

Some client device using 16bit index, and 4Bytes data with (Little Endian or Big Endian).

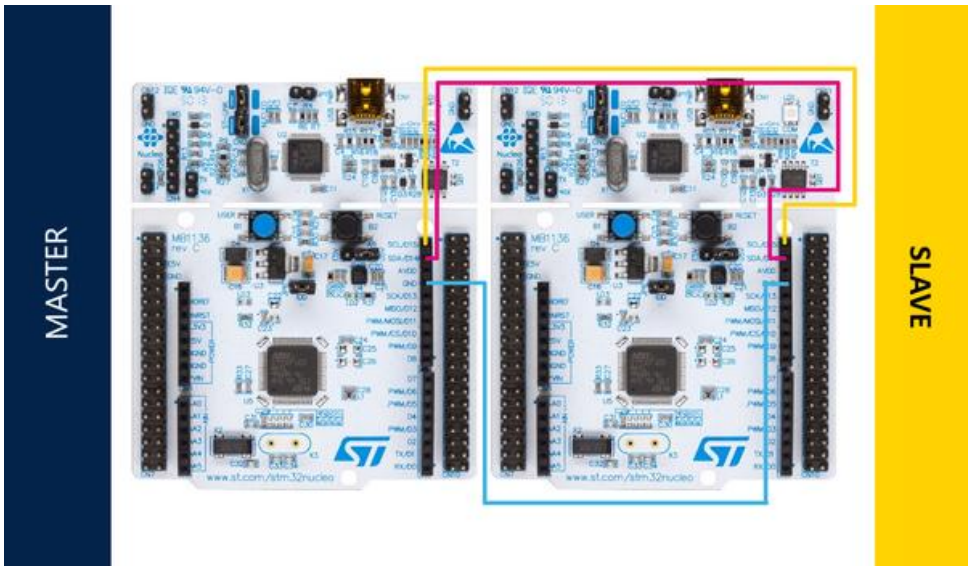
Some master (ex:Serdes) requests SCL delay before it checks Ack/Nack on the local clients device.

"Clock stretching"

Mode	Master functions	Slave or Client
Blocking	HAL_I2C_Master_Transmit() HAL_I2C_Master_Receive() HAL_I2C_Mem_Write() HAL_I2C_Mem_Read()	HAL_I2C_Slave_Transmit() HAL_I2C_Slave_Receive()
Non-blocking Interrupt Mode	HAL_I2C_Master_Transmit_IT() HAL_I2C_Master_Receive_IT() HAL_I2C_Mem_Write_IT() HAL_I2C_Mem_Read_IT()	HAL_I2C_Slave_Transmit_IT() HAL_I2C_Slave_Receive_IT()
Non-blocking DMA mode	HAL_I2C_Master_Transmit_DMA() HAL_I2C_Master_Receive_DMA() HAL_I2C_Mem_Write_DMA() HAL_I2C_Mem_Read_DMA()	HAL_I2C_Slave_Transmit_DMA() HAL_I2C_Slave_Receive_DMA()

Pin	Name

PB8	SCL
PB9	SDA
	GND
	3.3V



File -> New -> STM32 Project
Select NECLEO-L476GR.
Set Project Name and Location
Open STM32CubeMX
Select wiki.ioc
Under Connectivity, select I2C1
Select PB8: SCL and PB9: SDA for I2C1
Set the I2C Mode as I2C (standard mode) and the I2C Frequency at 100 KHz
In the GPIO Settings tab
  Set GPIO Pull-up/Pull-down to Pull-up
  Set Maximum output speed to Very High

<table>
<tr><td></td><td></td></tr>
<tr><td>Interrupt Mode only</td><td>In the NVIC settings tab
Enable interrupt</td></tr>
<tr><td>DMA Mode only</td><td>In the DMA settings tab, select Add.
<table>
<tr><td>DMA Request</td><td>Channel</td><td>Direction</td><td>Priority</td></tr>
<tr><td>I2C1_Rx</td><td>DMA Channel 7</td><td>Peripheral To Memory</td><td>Low</td></tr>
</table>
.</td></tr>
</table>

Click "***Ctrl+S***" to generate the project.

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t TX_Buffer [] = "A" ; // DATA to send
```

/* USER CODE END PV */

Blocking Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit(&hi2c1,20,TX_Buffer,1,1000); //Sending in Blocking mode
	HAL_Delay(100);
	/* USER CODE END 2 */

Interrupt Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit_IT(&hi2c1,20,TX_Buffer,1); //Sending in Interrupt mode
	HAL_Delay(100);
	/* USER CODE END 2 */

Dma Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit_DMA(&hi2c1,20,TX_Buffer,1); //Sending in DMA mode
	HAL_Delay(100);
	/* USER CODE END 2 */

On Rx (Client Board side)

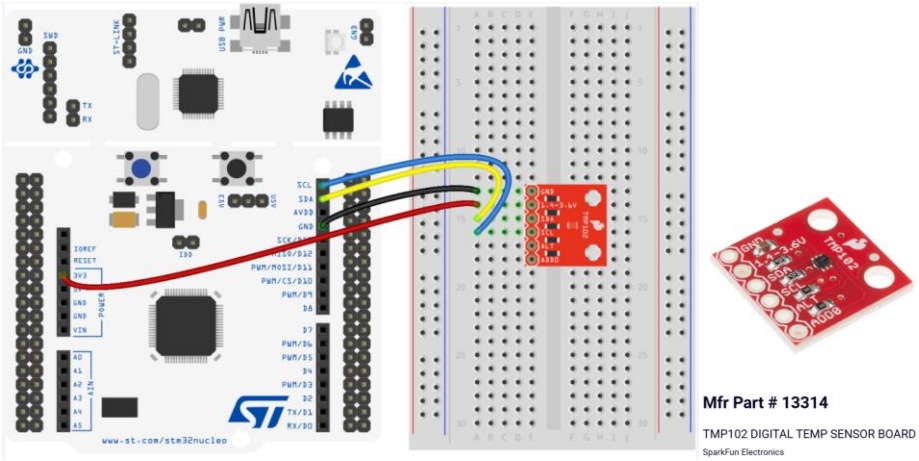
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t RX_Buffer [1] ; // DATA to receive
/* USER CODE END PV */

Blocking Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Slave_Receive(&hi2c1 ,(uint8_t *)RX_Buffer, 1,1000); //Receiving in Blocking mode
	HAL_Delay(100);
	/* USER CODE END 2 */

Interrupt Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Slave_Receive_IT(&hi2c1 ,(uint8_t *)RX_Buffer, 1); //Receiving in Interrupt mode
	HAL_Delay(100);
	/* USER CODE END 2 */

Dma Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Slave_Receive_DMA(&hi2c1 ,(uint8_t *)RX_Buffer, 1); //Receiving in DMA mode
	HAL_Delay(100);
	/* USER CODE END 2 */

4.1 with Temp Sensor Board



Under Connectivity, select I2C1
Select PB8: SCL and PB9: SDA for I2C1

Set the I2C Mode as I2C (standard mode) and the I2C Frequency at 100 KHz

In the GPIO Settings tab
Set GPIO Pull-up/Pull-down to Pull-up
Set Maximum output speed to Very High

Interrupt Mode only.
In the NVIC settings tab, Enable interrupt

Blocking Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit(&hi2c1,20,TX_Buffer,1,1000); //Sending in Blocking mode
	HAL_Delay(100);
	/* USER CODE END 2 */

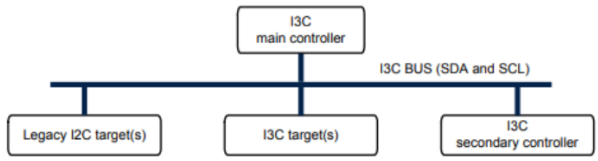
Interrupt Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit_IT(&hi2c1,20,TX_Buffer,1); //Sending in Interrupt mode
	HAL_Delay(100);
	/* USER CODE END 2 */

DMA Mode	/* USER CODE BEGIN 2 */
	HAL_I2C_Master_Transmit_DMA(&hi2c1,20,TX_Buffer,1); //Sending in DMA mode
	HAL_Delay(100);
	/* USER CODE END 2 */

4.2 I3C

An I^3C bus controller device drives the push-pull SCL line at the communication bus speed, up to 12.5 MHz.

The purpose of MIPI I^3C is threefold:
- Standardize communication inside embedded systems with a reconfigurable bus.
- Reduce the number of physical pins used.
- Support low-power, high-speed data transfer, up to 33 Mbps.



4.2.1 Features

The I^3C bus supports different modes and operations of various message types:

- Broadcast and direct common command code (CCC) messages to communicate with multiple devices or a specific one.
- Dynamic addressing: I^3C assigns a dynamic address, unlike I^2C, which has a static address.
- Private read/write transfers.
- Legacy I^2C messages: the I^3C controller can communicate with I^2C devices on the I^3C bus.
- In-band interrupt (IBI): the target device, connected to the bus, can send an interrupt to the controller over the two-wire (SCL/SDA).
- Hot-join request: the target can join the I^3C bus after initialization.
- Controller role request.

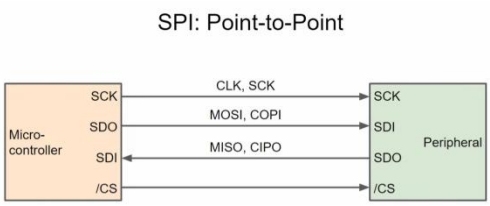
4.2.2 I2C vs I3C

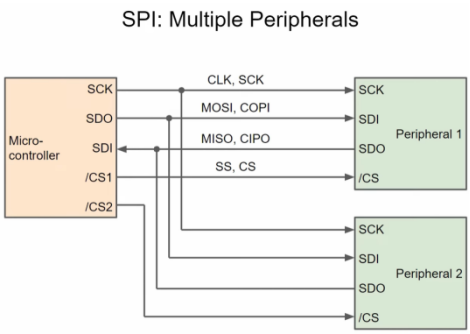
Feature	MIPI I3C	I2C
Bus speed	Up to 12.5 MHz	Up to 1 MHz
Signal	Open-drain & push-pull	Open-drain
Address	Dynamic address (7 bits) & static address (7 bits)	Static address (7 or 10 bits)
Interrupt	In-band interrupt	External I/O
Hot join	Yes	No
Common command codes (CCC)	Yes	No
Clock stretching	No	Yes
9th data bit	Transition bit	ACK or NACK
Start, restart & stop conditions	Similar to I2C	-

4.3 On Error

De not use retry!
Check I2C speed.
Connect Ground.
Check Pull-up register value and Ack transaction time.
Check Voltage, some I2C using 1.8V.

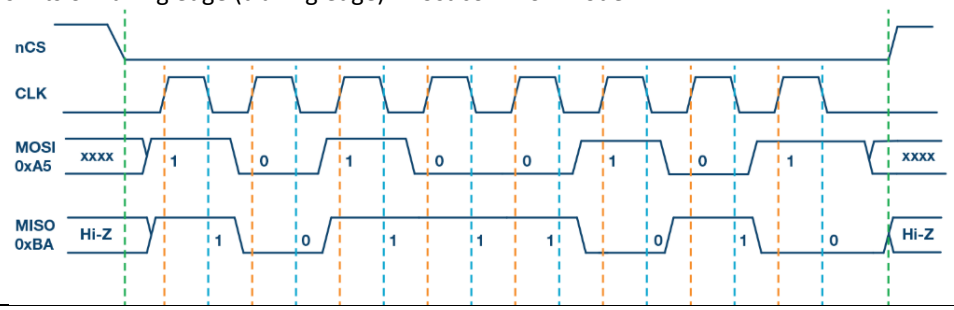
5 Peripheral – SPI





	Description
SCK	(Serial Clock) bus, controlled by the master device
MOSI	(Master Output Slave Input) bus
MISO	(Master Input Slave Output) bus
SS	Or CS(Chip Select). (Slave Select) bus, useful in the case of a multiple-slave configuration

Mode

	CPOL	CPHA	
0	0	0	Clock idles low, data sampled on rising edge (leading edge), shifts on falling edge (trailing edge). Most common mode. 
1	0	1	Clock idles low, data sampled on falling edge (trailing edge), shifts on rising edge (leading edge).
2	1	0	Clock idles high, data sampled on falling edge (leading edge), shifts on rising edge (trailing edge).
3	1	1	Clock idles high, data sampled on rising edge (trailing edge), shifts on falling edge (leading edge).

STM32 SPI modes

Full duplex

Half duplex

Receive only

Transmit only

Basic parameters:

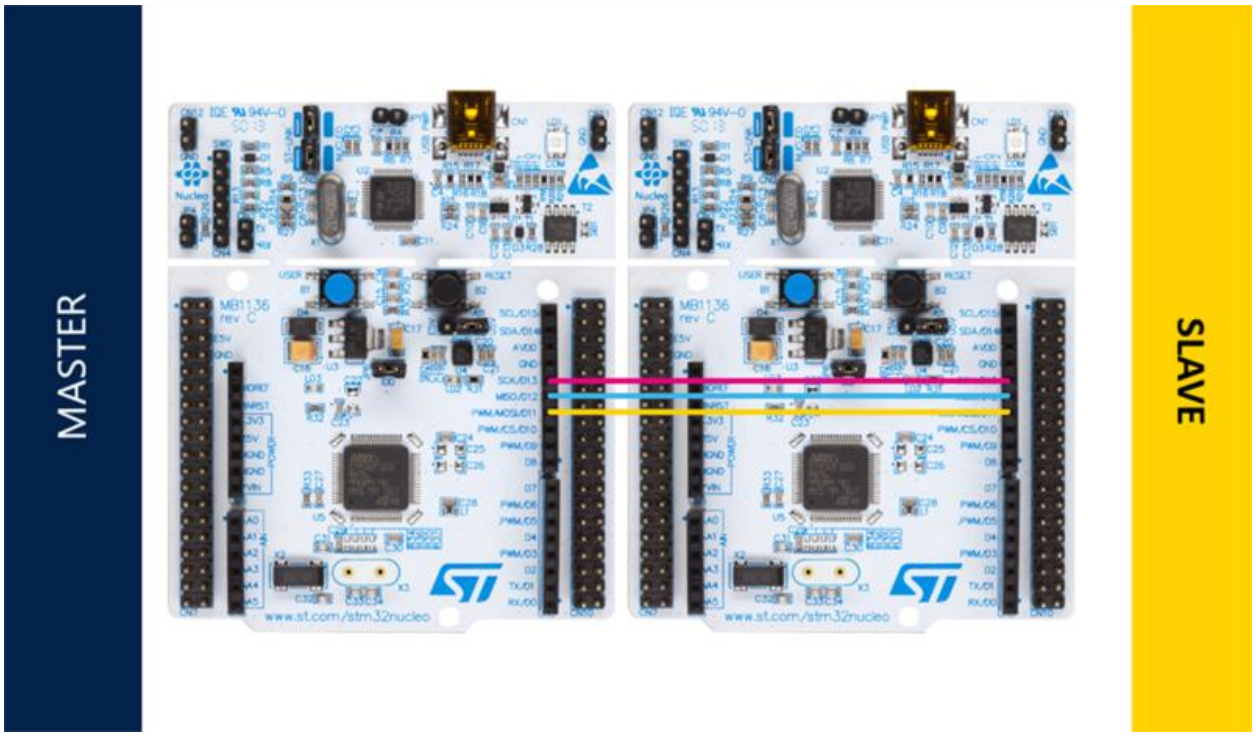
Frame format: Motorola or TI

Data size: 4 to 16 bits depending on STM32
First bit: MSB or LSB
Clock parameters:
Prescaler: from 2 to 256. Note that the SPI clock speed cannot exceed half the internal bus frequency.
Clock polarity (CPOL): low or high
Clock phase (CPHA): 1 or 2 edge

SPI HAL

	API Function
Blocking mode	HAL_SPI_Transmit() HAL_SPI_Receive() HAL_SPI_TransmitReceive()
Interrupt mode	HAL_SPI_Transmit_IT() HAL_SPI_Receive_IT() HAL_SPI_TransmitReceive_IT()
DMA mode	HAL_SPI_Transmit_DMA() HAL_SPI_Receive_DMA() HAL_SPI_TransmitReceive_DMA()

PA5	SCK
PA7	MOSI
PA6	MISO
	SS
	GND
	3.3V



5.1 SPI Master board

File->Nex->STM32 Project

Select the ***NUCLEO-L476RG*** board in the ***Board Selector*** tab as shown in the below figure.

Save the project.

Open the STM32CubeMX project from the workspace.

Under **Connectivity**, select **SPI1**.

Select **Full-Duplex Master** mode:

Pin PA5 as SCK

Pin PA6 as MISO

Pin PA7 as MOSI

Keep the default SPI settings, as shown below.

Basic	Format: Motorola Data Size: 4bits First Bit: MSB first
Clock Parameter	Prescaler(): 2 BaudRate: 40.0Mbps Clock Polarity (CPOL): Low Clock Phase (CPHA): 1 Edge
Advanced Parameters	CRC Calculation: Disabled NSSP Mode: Enabled NSS Signal Type: Software

Leave the GPIO settings in the default configuration, as shown below.

Pin Name	Signal on Pin	GPIO output	GPIO mode	GPIO Pull-up	Maximum	Fast Mode	User Label	Modified
PA5	SPI1_SCK							
PA6	SPI1_MISO							
PA7	SPI1_MOSI							

Interrupt mode only	Go to the NVIC settings tab. Enable *SPI1 global interrupt*
DMA mode only	Go to the DMA settings tab and click the *Add* button. DMA Request: SPI1_TX; Channel: DMA1 Channel 3; Direction: Memory to Peripheral; Priority: Low .

Push "***Ctrl+S***" to generate the project and click "Yes" in the popup window, as shown below.

```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t TX_Buffer [] = "A" ; // DATA to send
/* USER CODE END PV */
```

Blocking mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Transmit(&hspi1, TX_Buffer, 1, 1000); //Sending in Blocking mode
	HAL_Delay(100);
	/* USER CODE END 2 */

Interrupt mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Transmit_IT(&hspi1, TX_Buffer, 1); //Sending in Interrupt mode
	HAL_Delay(100);
	/* USER CODE END 2 */

DMA mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Transmit_DMA(&hspi1, TX_Buffer, 1);; //Sending in DMA mode
	HAL_Delay(100);
	/* USER CODE END 2 */

5.2 SPI client board

Under Connectivity, select SPI1.
Select Full-Duplex Slave mode:

Interrupt mode only	Go to the NVIC settings tab. Enable *SPI1 global interrupt*
DMA mode only	Go to the DMA settings tab and click the *Add* button. DMA Request | Channel | Direction | Priority SPI1_RX | DMA1 Channel 3 | Memory to Peripheral | Low .

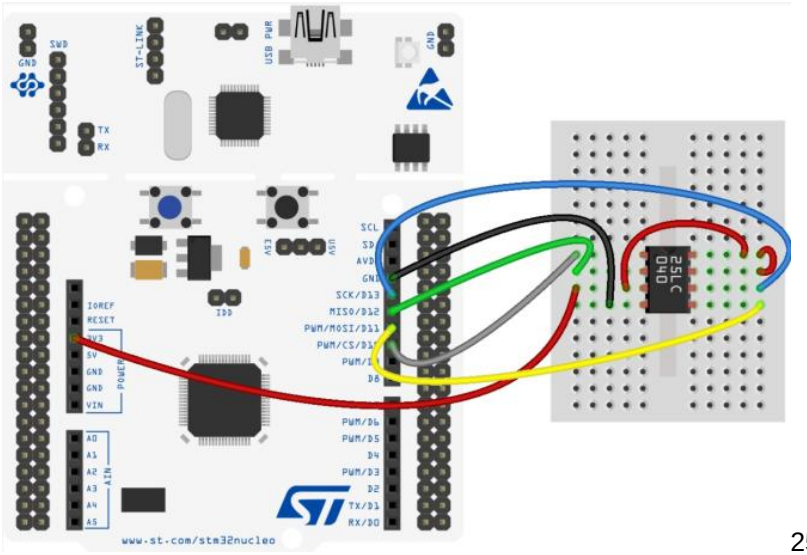
```
/* USER CODE BEGIN PV */
/* Private variables */
uint8_t RX_Buffer [1] ; // DATA to receive
/* USER CODE END PV */
```

Blocking mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Receive(&hspi1, RX_Buffer, 1, 1000); //Receiving in Blocking mode
	HAL_Delay(100);
	/* USER CODE END 2 */

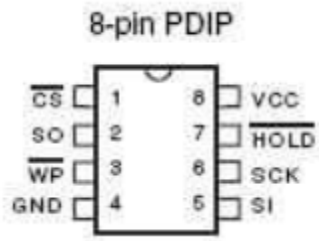
Interrupt mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Receive_IT(&hspi1, RX_Buffer, 1); //Receiving in Interrupt mode
	HAL_Delay(100);
	/* USER CODE END 2 */

DMA mode	/* USER CODE BEGIN 2 */
	HAL_SPI_Receive_DMA(&hspi1, RX_Buffer, 1); //Receiving in DMA mode
	HAL_Delay(100);
	/* USER CODE END 2 */

5.3 Serial EEPROM



25LC040 is 4K SPI EEPROM.



Pin Name	Function
$\overline{CS}$	Chip Select
SCK	Serial Data Clock
SI	Serial Data Input
SO	Serial Data Output
GND	Ground
VCC	Power Supply
$\overline{WP}$	Write Protect
$\overline{HOLD}$	Suspends Serial Input

AT25HP512 is a 65,536 byte serial EEPROM.

It supports SPI modes 0 and 3, runs at up to 10MHz at 5v and can run at slower speeds down to 1.8v.

memory is organized as 512 pages of 128 bytes each.

It can only be written 128 bytes at a time, but it can be read 1-128 bytes at a time.

Control registers	SPCR	control settings for various microcontroller functionalities. Usually each bit in a control register effects a particular setting, such as speed or polarity.
Data registers	SPDR	simply hold bytes. For example, the SPI data register (SPDR) holds the byte which is about to be shifted out the MOSI line, and the data which has just been shifted in the MISO line.
Status registers	SPSR	change their state based on various microcontroller conditions. For example, the seventh bit of the SPI status register (SPSR) gets set to 1 when a value is shifted in or out of the SPI.

SPI Control Register SPCR

Name	Bit	Description
SPIE	7	Enables the SPI interrupt when 1
SPE	6	Enables the SPI when 1
DORD	5	Sends data least Significant Bit First when 1, most Significant Bit first when 0
MSTR	4	Sets the Arduino in controller mode when 1, peripheral mode when 0

CPOL	3	Sets the data clock to be idle when high if set to 1, idle when low if set to 0
CPHA	2	Samples data on the falling edge of the data clock when 1, rising edge when 0
SPR1 and SPR0	1 0	Sets the SPI speed, 00 is fastest (4MHz) 11 is slowest (250KHz)

opcode	number	Description
WREN	6	Write Enable
WRDI	4	Reset the write enable latch (disable write operations)
RDSR	5	Read STATUS register instruction provides access to the STATUS register. The STATUS register may be read at any time, even during a write cycle.
WRSR	1	Write STATUS register instruction allows the user to select one of four levels of protection for the array by writing to the appropriate bits in the STATUS register. The array is divided up into four segments. The user has the ability to write-protect none, one, two, or all four of the segments of the array.
READ	3	Read data from memory array beginning at selected address
WRITE	2	Write data to memory array beginning at selected address

Status Register

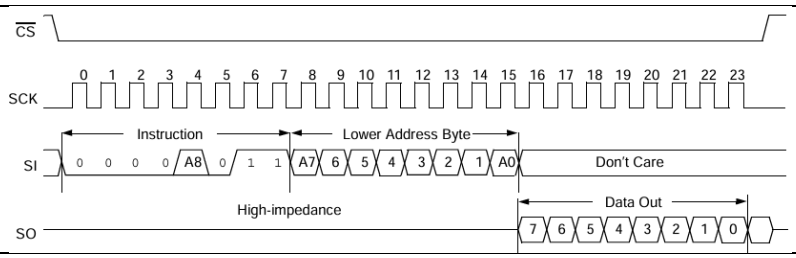
Bit7	6	5	4	3	2	1	0
x	x	x	x	BP1	BP0	WEL	WIP

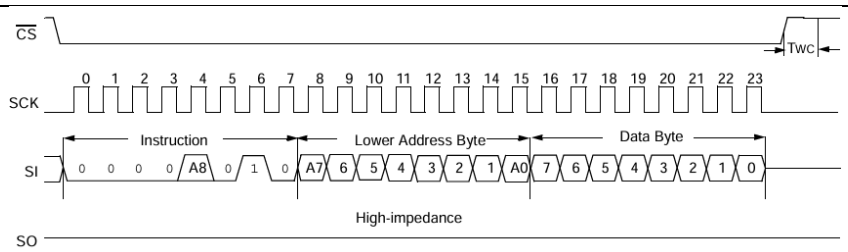
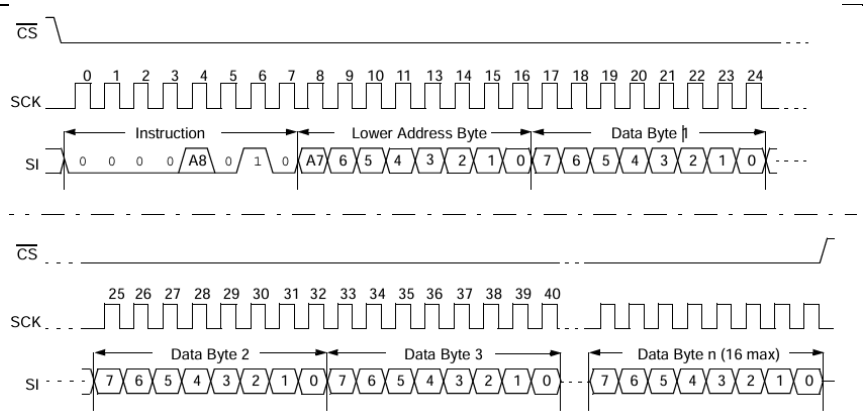
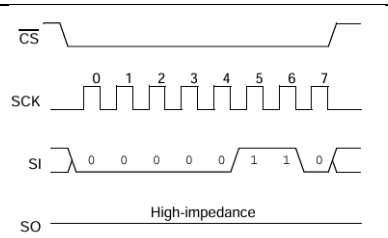
WIP	The Write-In-Process bit indicates whether the 25XX040 is busy with a write operation. When set to a ‘1’, a write is in progress, when set to a ‘0’, no write is in progress. This bit is read-only.
WEL	The Write Enable Latch bit indicates the status of the write enable latch. When set to a ‘1’, the latch allows writes to the array, when set to a ‘0’, the latch prohibits writes to the array. The state of this bit can always be updated via the WREN or WRDI commands regardless of the state of write protection on the STATUS register. This bit is read-only.
BP0 and BP1	The Block Protection bits indicate which blocks are currently write-protected. These bits are set by the user issuing the WRSR instruction. These bits are nonvolatile.

BP1	BP0	Array Addresses Write-Protected
0	0	none
0	1	upper 1/4 (0180h-01FFh)
1	0	upper 1/2 (0100h-01FFh)
1	1	all (0000h-01FFh)

.

Read Sequence



Byte Write Sequence	
Page Write Sequence	
Write Enable Sequence	

Sample Code for AT25HP512

	Sample Code
define	#define DATAOUT 11//COPI #define DATAIN 12//CIPO #define SPICLOCK 13//sck #define CHIPSELECT 10//cs //opcodes #define WREN 6 #define WRDI 4 #define RDSR 5 #define WRSR 1 #define READ 3 #define WRITE 2

variable	byte eeprom_output_data;
	byte eeprom_input_data=0;
	byte clr;
	int address=0;
	//data buffer
	char buffer [128];

Fill 128 data	/* We will fill data from 0 to 127. */ void fill_buffer()

	{
	for (int i=0; i<128; i++)
	buffer[i]=i;
	}

SPI MOSI and MISO	char spi_transfer(volatile char data)
	{
	SPDR = data; // Start the transmission
	while (!(SPSR & (1<<SPIF))) {}; // Wait for the end of the transmission
	return SPDR; // return the received byte
	}

SETUP and Write Example	

SETUP and Write Example	/* write 128 data */
	void setup()
	{
	
	//Serial.begin(9600);
	
	pinMode(DATAOUT, OUTPUT);
	pinMode(DATAIN, INPUT);
	pinMode(SPICLOCK,OUTPUT);
	pinMode(CHIPSELECT,OUTPUT);
	digitalWrite(CHIPSELECT,HIGH); //disable device

Set SPCR	// SPCR = 01010000
	//interrupt disabled,spi enabled,msb 1st,controller,clk low when idle,
	//sample on leading edge of clk,system clock/4 rate (fastest)
	SPCR = (1<<SPE)|(1<<MSTR);
	clr=SPSR;
	clr=SPDR;
	delay(10);

Write Enable	digitalWrite(CHIPSELECT,LOW);
	spi_transfer(WREN); //write enable
	digitalWrite(CHIPSELECT,HIGH);

Update buffer	fill_buffer();

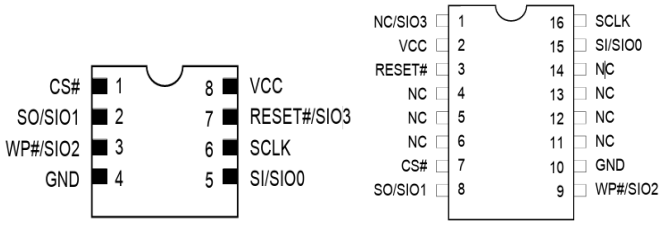
Write 128 Data	delay(10);
	digitalWrite(CHIPSELECT,LOW);
	spi_transfer(WRITE); //write instruction
	
	address=0; /* TODO */
	spi_transfer((char)(address>>8)); //send MSByte address first
	spi_transfer((char)(address)); //send LSByte address
	
	//write 128 bytes
	for (int i=0; i < 128; i++) {
	spi_transfer(buffer[i]); //write data byte
	}
	digitalWrite(CHIPSELECT,HIGH); //release chip
	
	//wait for eeprom to finish writing
	delay(3000);
	}

Read data example	

Read Data	byte read_eeprom(int EEPROM_address)
	{
	  int data;
	
	  digitalWrite(CHIPSELECT,LOW);
	  spi_transfer(READ); //transmit read opcode
	  spi_transfer((char)(EEPROM_address>>8)); //send MSByte address first
	  spi_transfer((char)(EEPROM_address)); //send LSByte address
	  data = spi_transfer(0xFF); //get data byte
	  digitalWrite(CHIPSELECT,HIGH); //release chip, signal end transfer
	
	  return data;
	}

5.4 SPI Flash Memory

MX25L12845G



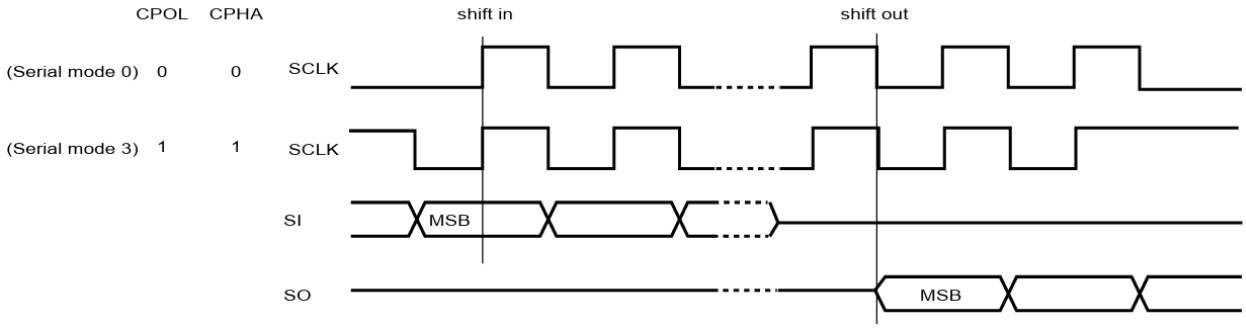
Pin Description

SYMBOL	DESCRIPTION
CS#	Chip Select
SI/SIO0	Serial Data Input (for 1 x I/O)/ Serial Data Input & Output (for 2xI/O or 4xI/O read mode)
SO/SIO1	Serial Data Output (for 1 x I/O)/ Serial Data Input & Output (for 2xI/O or 4xI/O read mode)
SCLK	Clock Input
WP#*/SIO2	Write Protection Active Low or Serial Data Input & Output (for 4xI/O read mode)
RESET#*/SIO3	Hardware Reset Pin Active low or Serial Data Input & Output (for 4xI/O read mode)
NC/SIO3	NC or Serial Data Input & Output (for 4xI/O read mode)
RESET#*	Hardware Reset Pin Active low
VCC	+ 3V Power Supply
GND	Ground
NC	No Connection

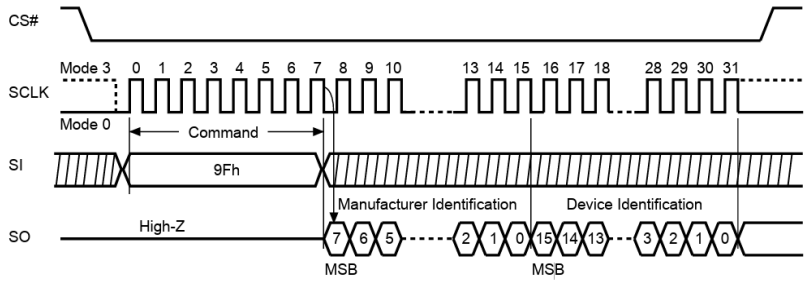
Command	Hex	
Read	03	Normal read
Fast Read	0B	
PP	02	Page Program, write
SE	20	Sector Erase
BE	D8	
CE	60/C7	Chip Erase
WREN	06	Write Enable
WRDI	04	Write Disable
RDSR	05	Read Status Register
RDCR	15	Read Configuration Register

WRSR	01	Write Status/Configuration register
WPSEL	68	Write Protection Selection
EQIO	35	Enable QPI
RSTQIO	F5	Reset QPI
..		
RDID	9F	Read Identification
..		

Serial Modes Supported



RDID

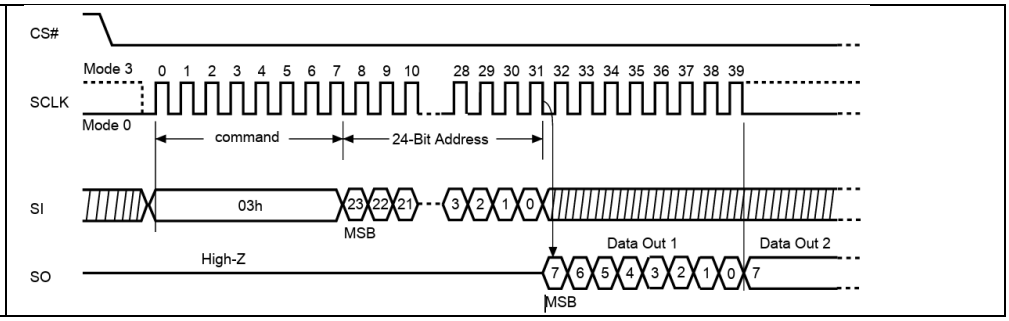


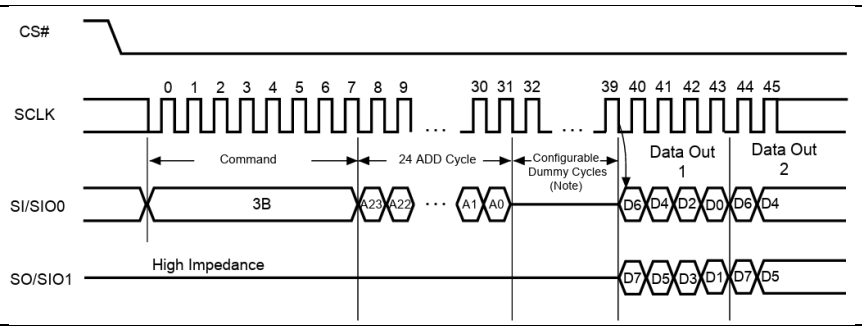
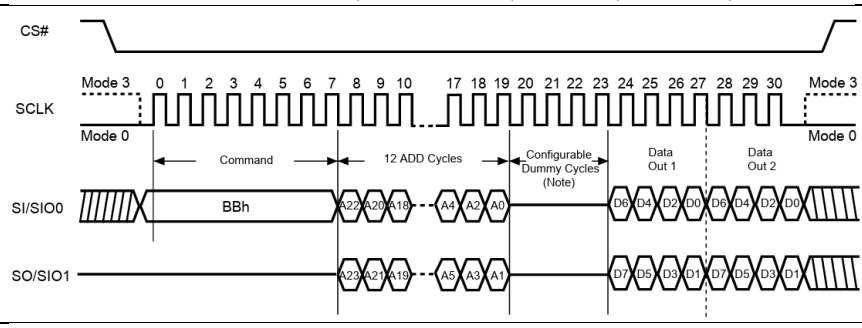
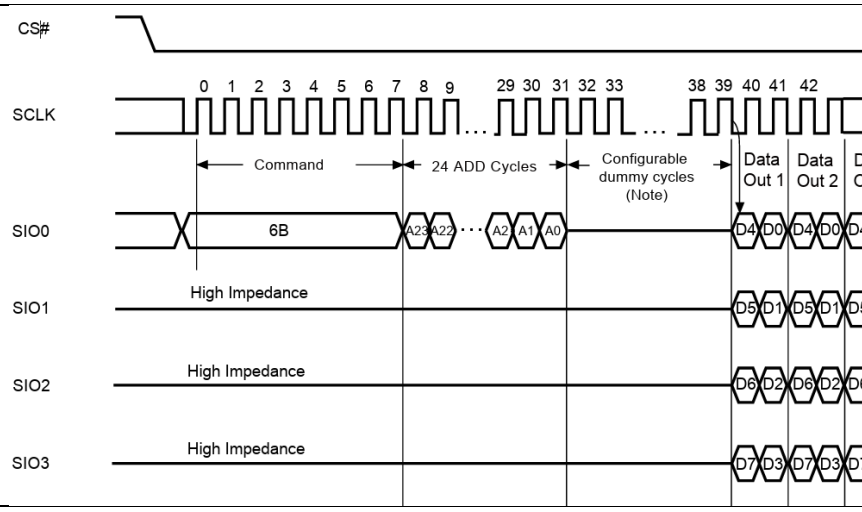
Status Register

Bit 7	6	5	4	3	2	1	0
SRWD	QE	BP3	BP2	BP1	BP0	WEL	WIP

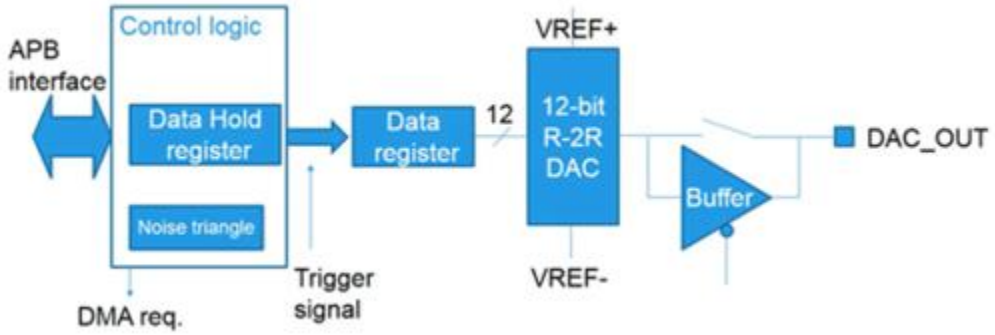
Configuration Register

Bit 7	6	5	4	3	2	1	0
DC1	DC0	Reserved	PBE	TB	Reserved	ODS1	ODS0

Read	

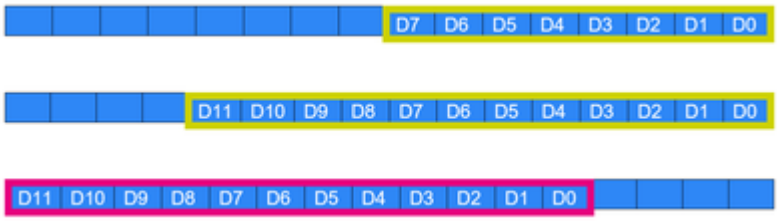
Dual Read	
2xIO Read	
QREAD	

6 Peripheral – DAC



6.1 DAC main features

8 or 12 bit mode: The DAC can support different input formats:
- **8-bit** right aligned data format (in yellow in the image below).
- **8-bit + 8-bit** in dual mode, to provide input data for two DACs.
- **12-bit** mode (in red in the image below): either in right or left aligned mode.



Buffered output: the DAC output can have a low impedance buffer to drive external loads.
Sample and Hold mode to reduce power consumption.
Synchronized update capability.
DMA capability to offload CPU.
Several trigger inputs: a timer, an external trigger, or a software trigger can update the DAC output data.
Signal generation: DAC integrates small logic to generate noise waves as well as triangle waves.

6.2 Configure DAC to generate a wave

File > New > STM32 Project
Select **NUCLEO-L476RG** board using **Board selector** as shown in the figure below:
Save the project: as DAC_xxx,

Search for **DAC1** in the left search field.
We will use **PA5** as the DAC's output.
As it is configured as GPIO Output by default, we need to reset it first.

Connect OUT2 **only to external pin**. Notice how **PA5** is automatically set to DAC1_OUT2 on the pinout view. **Parameter Settings** should be set up as in the below example: **normal mode**, **buffer enabled** and **no trigger**.

Click **Ctrl+S** to generate the project:

6.3 Add code

```
/* USER CODE BEGIN PV */
uint16_t dac_value=0;
/* USER CODE END PV */
```

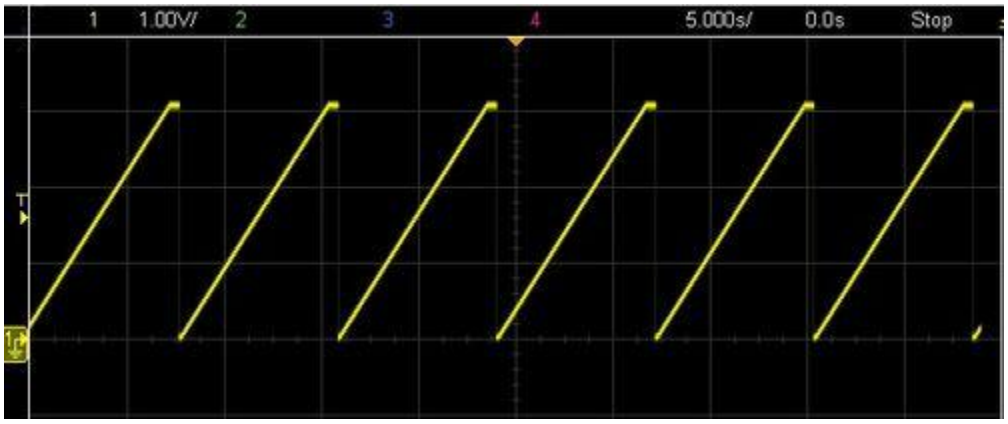
```
/* USER CODE BEGIN 2 */
HAL_DAC_Start(&hdac1, DAC_CHANNEL_2);
/* USER CODE END 2 */
```

```
/* USER CODE BEGIN 3 */
HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_2, DAC_ALIGN_12B_R, dac_value);
if (dac_value < 4095) {
        dac_value++;
} else {
        dac_value=0;
}
HAL_Delay(1);
/* USER CODE END 3 */
```

Inside the *if* conditional statement, we check if **dac_value** is lower than **4095** (0xFFF) which is the maximum output value for a 12-bit resolution DAC:

- If the value is lower than the maximum value, it is incremented.
- If not, it is reset back to zero.
- Then, we add a 1ms delay in order to have a stable output value.

This will be running in the infinite loop. Therefore, the code will sweep through all the possible DAC output values, and then reset it to zero restarting the process all over again.



7 Peripheral – ADC and DMA

feature on STM32L4x6

ADC units	Up to 3 modules
Input channel	Up to 24 external channels (GPIOs), single/differential
Technology	12-bit successive approximation
Conversion time	188 nS, 5.33 Mega samples/s (when fADC_CLK = 80 MHz)
Functional mode	Single, Continuous, Scan, Discontinuous, or Injected
Triggers	Software or external trigger (for Timers & IOs)
Special functions	Hardware oversampling, analog watchdogs
Data processing	Interrupt generation, DMA requests
Low-power modes	Deep power-down, auto delay, power consumption dependent on speed

7.1 sample code

Create a simple application to start the ADC and measure the DAC output.

File->New->STM32 Project
Select NUCLEO-L476RG board using Board selector as shown in the figure below:
Project Name as ADC_xxx and finish

7.2 Configure DAC

We will use **PA5** as the DAC's output.
Search for **PA5** in the bottom right search field, then click **Reset_State**.

Search for **DAC1** in the left search field.
OUT2 connected to **only to external pin**. Notice how **PA5** is automatically set to DAC1_OUT2 on the pinout view.
Parameter Settings should be set up as in the below example: **normal mode**, **buffer enabled** and **no trigger**.

7.3 Configure ADC

Configure ADC with the following settings:
- Asynchronous clock mode div by **256**
- Resolution 12bits
- Right alignment
- Continuous conversion mode **Enabled**
- Software trigger
- sampling time for CH6 (92.5cycles)
- DMA Continuous requests **Enabled**

7.4 Configure DMA

The ADC conversion values will be transferred into memory by DMA controller.

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	Peripheral To Memory	Low

Mode: Circular
Data Width: Half Word Half Word
Click **Ctrl+S** to generate the project:

7.5 Add Code

```
/* USER CODE BEGIN PV */
uint32_t value_adc;
uint32_t value_dac=0;
/* USER CODE END PV */
```

```
/* USER CODE BEGIN 2 */
HAL_DAC_Start(&hdac1,DAC_CHANNEL_2);
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
HAL_ADC_Start_DMA(&hadc1,(uint32_t*)&value_adc,1);
/* USER CODE END 2 */
```

```
/* USER CODE BEGIN 3 */
while (1)
{
  HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_2, DAC_ALIGN_12B_R, value_dac);
  value_dac++;
  if(value_dac>4095) {
    value_dac=0;
  }
}
/* USER CODE END 3 */
```

First, DAC is configured as input voltage source for ADC. It uses output channel 2 (OUT2) and is connected to an external pin (PA5).
The ADC is configured to use asynchronous clock mode with a clock prescaler of 256 ensuring that ADC is not running too fast.
ADC is configured to have a resolution of 12 bits, right alignment, continuous conversion mode enabled (in order to convert data continuously), and software trigger with a sampling time of 92.5 cycles for channel 6 (**PA1**).
The ADC values are transferred to memory using DMA controller, which is configured as follows :
- Circular mode, allowing an updated indefinitely transfer.
- Continuous DMA request enabled allowing an uninterrupted transfer of data to the memory.
ADC and DAC should have **similar** values, while using PA1 – PA5 jumper.
Therefore, the code will sweep through all the possible ADC output values, and then reset it to zero restarting the process all over again.

8 Peripheral – HRTIM PWM Motor Control

8.1 Configure HRTIM

In the Pinout & Configuration window, select HRTIM1 and choose TA1 output active.
Check the PA8 pin is declared correctly on the Pinout view on the right.
In **User Constants**, click on **add** and enter this constant name **TIMA_DUTY_CYCLE** with the value **0.5**.
In **Timer A**, in section **Time Base Setting** , observe the configuration: **up_counting** and **continuous** mode with a resulting PWM Frequency of 83050 Hz (Period = 0xFFDF). Keep the settings.
Scroll down until **Compare Unit 1**, and fill as below :
1. Enable Compare Unit 1 Configuration
2. Enter **0xFFDF * TIMA_DUTY_CYCLE** in the Compare Value
3. Keep **Timer Compare 1 event is generated when counter is equal**
Scroll down until **Output 1 Configuration** and fill as below:
1. Enter **1** as number of Active Set Source and select **Timer period event forces the output to its active state** as the first set source
2. Enter **1** as number of Active Reset Source and select **Timer compare 1 event forces the output to its inactive state** as the first reset source
Click ***Ctrl+S*** to generate the project.

8.2 Add Code

```
/* USER CODE BEGIN 2 */

/* Enable HRTIM's outputs TA1 and start Timer A */
// Enable the generation of the waveform signal on the designated output
HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TA1);
// Start the counter of the Timer A operating in waveform mode
HAL_HRTIM_WaveformCounterStart(&hhrtim1, HRTIM_TIMERID_TIMER_A);
/* USER CODE END 2 */
```

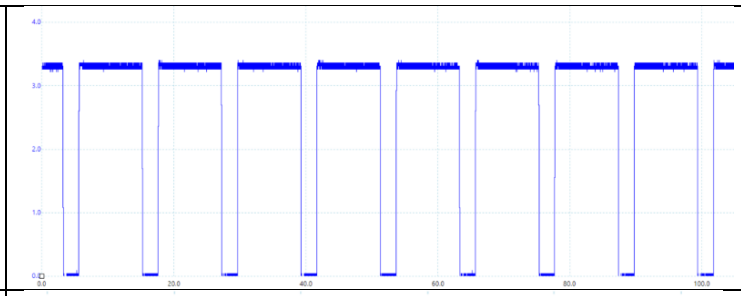
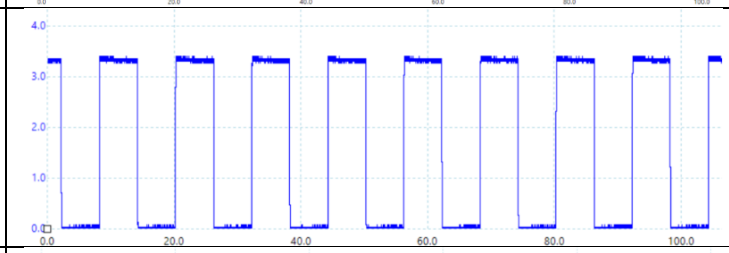
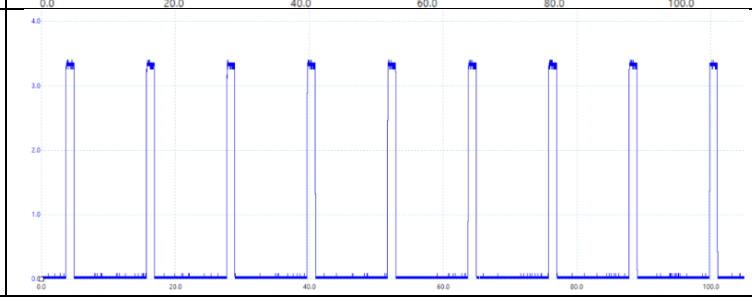
8.3 Result

If you want to observe the output PWM, you need to check on the **CN9** the pin n°8 named **D7** with an oscilloscope.
For a duty cycle of **0.5**, we have the waveform below:

Open and edit ***main.h***.
In the private defines section, change the value of **TIMA_DUTY_CYCLE** such as the picture below.

```
/* Private defines ------------------------------------*/
#define TIMA_DUTY_CYCLE 0.8
```

TIMA_DUTY_CYCLE	

0.8	
0.5	
0.1	

8.4 To go further about DMA burst

The DMA burst controller allows:
- Update multiple registers (such as the CMP1xR to control the duty cycle of the PWM).
- Reprogram dynamically one or several timing units.

Go back in CubeMX in the **HRTIM1** configuration in **Timer A** / **Time Base Setting** :
- Enable the Burst DMA configuration.
- Enter **1** in the number of register to update.
- Choose **CMP1xR register is updated by Burst DMA accesses** as the first register.

Then, go in the **DMA settings** window, and configure as shown in the picture below:
- Click on **Add**.
- Select **HRTIM1_A** in DMA request.
- Uncheck Increment address boxes (for both peripheral and memory).
- Choose **Word** for data width (for both peripheral and memory).

The last changes in the **DMA settings** window created a warning in **Timer A** / **Timing Unit** / **Number of Timer A Internal DMA request sources**. Enter **1** instead of 0 like in the picture below :

```
/* USER CODE BEGIN 2 */

/* Enable HRTIM outputs TA1 and start Timer A */
// Enable the generation of the waveform signal on the designated output
HAL_HRTIM_WaveformOutputStart(&hhrtim1, HRTIM_OUTPUT_TA1);
// Start the counter of the Timer A operating in waveform mode
```

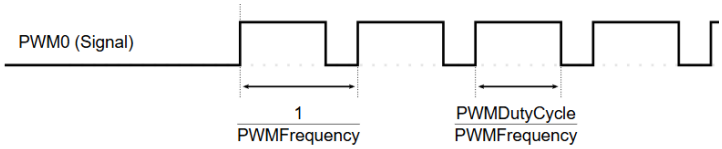
```
HAL_HRTIM_WaveformCounterStart(&hhrtim1, HRTIM_TIMERID_TIMER_A);

static uint32_t Reg_Update [] =
{
  0xFFA0,
};
 //Add a breakpoint here
if(HAL_HRTIM_BurstDMATransfer(&hhrtim1,
 HRTIM_TIMERINDEX_TIMER_A,
 (uint32_t) Reg_Update, sizeof(Reg_Update)/sizeof(uint32_t)) != HAL_OK)
{
  Error_Handler();
}

/* USER CODE END 2 */
```

HAL_HRTIM_BurstDMAConfig(): configures the burst DMA burst controller. It allows having multiple HRTIM registers updated with a single DMA request. The burst DMA operation is started by calling HAL_HRTIM_BurstDMATransfer().
For the last step, go in the function **HAL_HRTIM_MspInit()** where the HRTIM1 DMA is initialize. Declare the direction to **DMA_MEMORY_TO_MEMORY** and not DMA_MEMORY_TO_PERIPH.

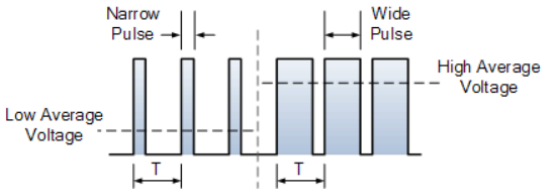
```
hdma_hrtim1_a.Init.Direction = DMA_MEMORY_TO_MEMORY;
```

"The DMA channels may operate without being triggered by a request from a peripheral. This mode is called memory-to-memory mode, and is initiated by software." (extract from chapter DMA in RM0440)

8.5 PWM Motor control



Pulse Width Modulated Waveform



Pulse Width Modulation (PWM) is an easy way of controlling DC motors and other such devices by using a digital signal that rapidly switching the supplied power “ON” and “OFF”.
Duty cycle is commonly expressed as a percentage. So a 10% duty cycle means the signal is ON 10% of the time, while a 90% duty cycle means it’s ON for 90% of the time and OFF for 10%. Clearly, a 50% duty cycle means the signal is ON for half the time and OFF for the other half producing a square wave signal.

Core Principles

Duty Cycle	The percentage of time the signal is "on" (high) versus "off" (low) in one cycle. A 10% duty cycle is slow, 90% is fast.
Average Voltage	The motor responds to the average voltage (Vavg = Vmax * Duty_Cycle), enabling efficient speed control.
Switching Frequency	High-frequency switching (typically > 10kHz) ensures smooth operation and eliminates audible noise

Common PWM Control Circuit Components

PWM Source	MCU
Switching Device	A power MOSFET (e.g., IRFZ44N) is required to handle higher motor currents, as the controller itself cannot supply enough power.
Freewheeling Diode (Flyback Diode)	A Schottky or fast-recovery diode (e.g., 1N5819 or 1N4007) placed in parallel with the motor (reverse biased) to protect the MOSFET from voltage spikes.
Gate Resistor	100Ohm to 1KOhm resistor connected to the MOSFET gate to manage switching speed and limit current.

Since you are familiar with the **V4L2** and **Linux kernel** world, you'll find that while the concepts of PWM are the same, the implementation in a "bare-metal" or RTOS environment like STM32 is much more direct. You aren't dealing with a driver abstraction layer; instead, you are configuring the hardware timers (TIM) directly.
The following example uses the **STM32 HAL (Hardware Abstraction Layer)**, which is the most common way to write portable STM32 code.

1. The Logic
To generate PWM, a Timer counts from 0 to an **Auto-Reload Register (ARR)** value. A **Capture Compare Register (CCR)** sets the threshold:
- When the counter is below CCR, the signal is **High**.
- When the counter is above CCR, the signal is **Low**.
2. Sample Code (HAL)
This snippet assumes you have configured **Timer 3, Channel 1** (usually mapped to a pin like PA6 or PB4) using CubeMX.

```
/* Private variables ---------------------------------------------------------*/
// Assuming a 72MHz Internal Clock
// To get a 1kHz PWM frequency:
// Period (ARR) = 7199
// Prescaler = 9
// Frequency = 72,000,000 / [(7199+1) * (9+1)] = 1000 Hz

uint32_t dutyCycle = 0;

int main(void)
{
  // 1. Initialize Hardware Abstraction Layer and System Clock
  HAL_Init();
  SystemClock_Config();

  // 2. Initialize Timer (Generated by CubeMX)
  MX_TIM3_Init();

  // 3. Start PWM on Timer 3, Channel 1
  HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
```

```
  while (1)
  {
    // Gradually increase brightness (0% to 100% duty cycle)
    for (dutyCycle = 0; dutyCycle <= 7200; dutyCycle += 100)
    {
      // Update the Compare Register (CCR) to change Duty Cycle
      __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, dutyCycle);

      HAL_Delay(50);
    }
  }
}
```

3. Key Register Calculations

When you move away from the GUI and need to calculate values for your header files or initialization structs:

$$Frequency = \frac{TimerClock}{(Prescaler + 1) \times (Period + 1)}$$

$$DutyCycle\% = \frac{CCR}{ARR + 1} \times 100$$

4. Direct Register Access (LL Drivers)

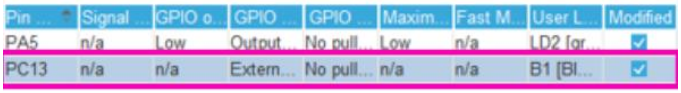
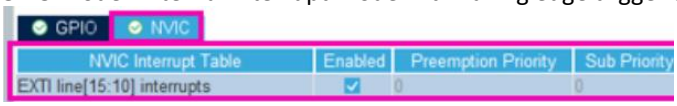
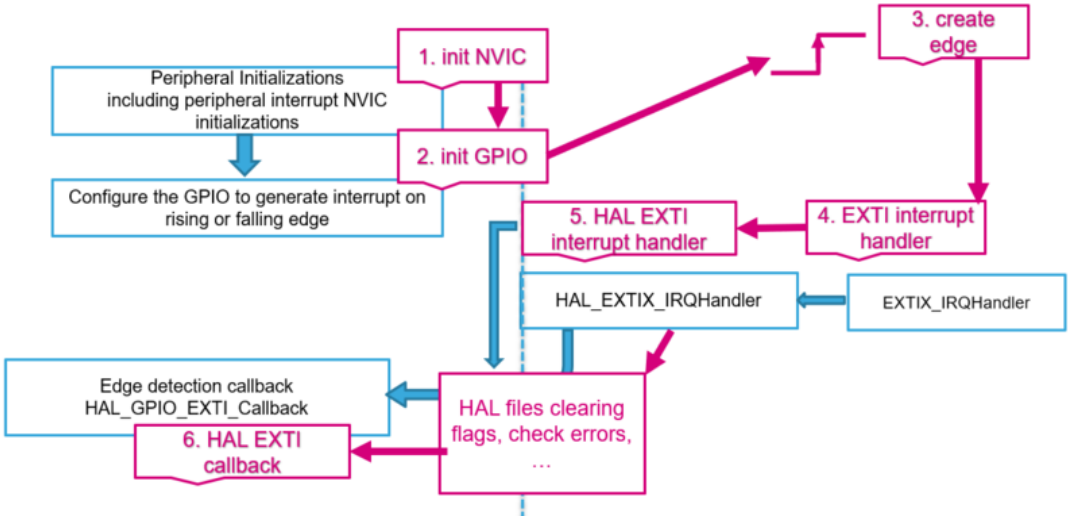
If you prefer a "thinner" approach closer to the metal (similar to writing a custom Linux driver without the overhead of the full HAL), you would use the **Low-Layer (LL)** macros:

```
// Set Duty Cycle directly via the Capture Compare Register
LL_TIM_OC_SetCompareCH1(TIM3, 3600); // 50% duty if ARR is 7199
```

Professional Considerations

- **Complementary Outputs:** If you are working on motor control (like H-bridges), look into "Advanced Timers" (TIM1/TIM8), which support dead-time insertion to prevent short circuits.
- **DMA Integration:** For dynamic waveforms (like generating audio or complex LED patterns), you can use DMA to update the CCR register automatically every timer cycle without CPU intervention.
- **Resolution:** If you need extremely high-resolution PWM (e.g., for power conversion), the **STM32G4** or **H7** series feature a "High-Resolution Timer" (HRTIM) capable of picosecond precision.

9 other peripherals

	Description
GPIO	PA5 with LED
UART	PA9 USART1_TX, PA10 USART1_RX
SPI	PA5:SCK, PA6:MISO, PA7 MOSI, SS:CS(??)
I2C / I3C	I2C PB8:SCL, PB9:SDA I3C PB6:SCL PB7:SDA
DAC	Generate signal on PA5
ADC	Read signal on PA1 that generated from DAC on PA5. It using PA1 – PA5 jumper.
HRTIM	**HRTIM** stands for high-resolution timer. HRTIM is a highly accurate timer that can generate complex waveforms such as **PWM**, phase-shifted. It is made up of six 16-bit up counter and a controller timer with very highly fine timing resolution
EXTI	External Interrupt/event controller. consists of up to 40 edge detectors for generating event/interrupt requests on STM32L47x/L48x devices. Each input line can be independently configured to select the type (interrupt or event) and the corresponding trigger event (rising, falling, or both). Objective: - Configure the GPIO that is connected to the user Button as External Interrupt (EXTI) with falling edge trigger using STM32CubeIDE - Learn how to configure the Interrupt Controller : the NVIC - Verify the correct functionality by pressing a button that turns on a LED Configure the LED pin as GPIO_Output (**PA5** on NUCLEO-L476RG). Configure a button pin as GPIO_EXTI (**PC13** on NUCLEO-L467RG).  GPIO mode: External Interrupt Mode with Falling edge trigger detection.   Code

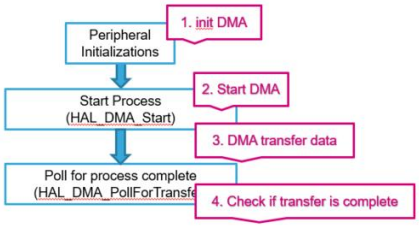
<table>
<tr><td></td><td>

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  if(GPIO_Pin == GPIO_PIN_13) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
  } else {
      __NOP();
  }
}
/* USER CODE END 4 */
```

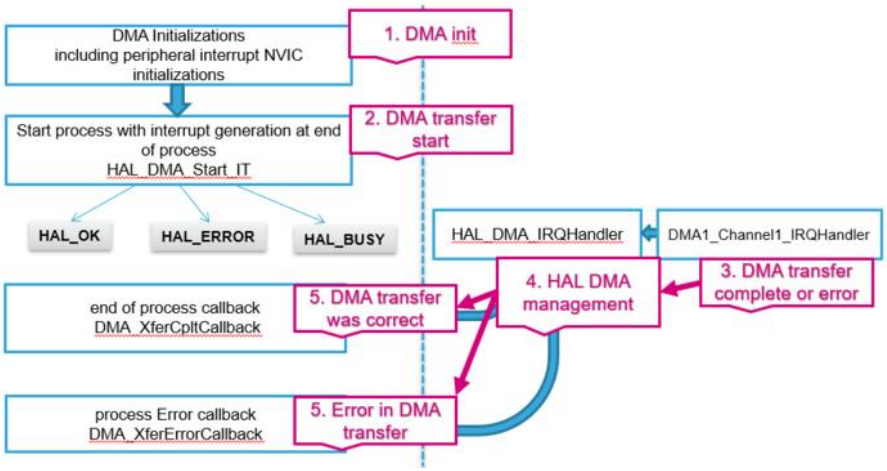
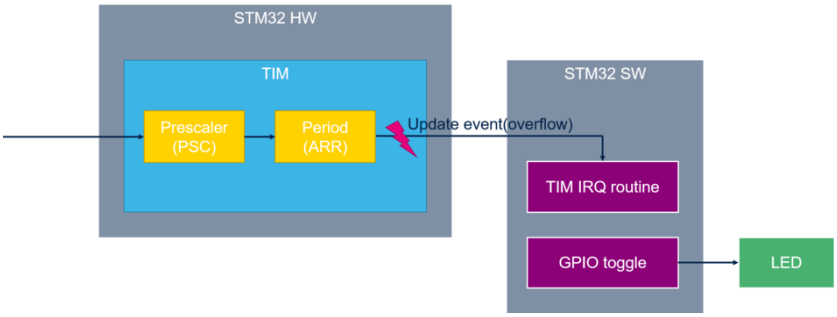
.
</td></tr>
<tr><td>PWR</td><td>low-power modes

- Sleep mode
- Low-power run mode
- Low-power sleep mode
- Stop 0, Stop1, Stop2 modes
- Standby mode
- Shutdown mode</td></tr>
<tr><td>LPBAM</td><td>The LPBAM stands for low-power background autonomous mode.
available in the STM32U5 products series</td></tr>
<tr><td>DMA</td><td>This article uses the STM32L476 device as an example. The STM32L476 device embeds 2 DMAs: DMA1 and DMA2.

<table>
<tr><th>DMA features</th><th>DMA1</th><th>DMA2</th></tr>
<tr><td>Number of regular channels</td><td>7</td><td>7</td></tr>
</table>



DMA with Interrupt</td></tr>
</table>

	
TIM	built-in timers General-purpose timers Advanced timers Basic timers Low-power timers High-resolution timers 
RTC	Real Time Clock A RTC is an independent binary-coded decimal (BCD) timer/counter. The RTC provides a time-of-day clock/calendar with programmable alarm interrupt. In addition, the RTC can provide an automatic wakeup to manage all low-power modes.
WDG	watchdog The main goal of this IP is to detect and resolve malfunctions due to software failures.
PLAY	Programmable Logic ArraY is a new peripheral that allows you to add custom logical functions or state machines on existing signals, without requiring an external FPGA or additional logic ICs.